

PRODUCT OVERVIEW AND SECURITY ARCHITECTURE

ODRE PQC v0.2.9

Application-Embedded Post-Quantum Security Boundary

Integration experience · Security boundary · Operations

ENGLISH EDITION

September 2026

Document map

01	1. Product Overview
02	2. Minimal Integration Experience
03	3. Product Architecture
04	4. Fail-Closed Security Contract
05	5. Platform Adapter and Native Runtime
06	6. Operational Diagnosis
07	7. Worker and Lifecycle Architecture
08	8. Trust Boundaries and Attack Surface
09	9. Deployment Architecture
10	10. Product Identity
11	11. Conclusion

Product Overview and Security Architecture

Application-Embedded Post-Quantum Security Boundary

ODRE PQC is a cross-platform security product designed to place a verifiable protection boundary in front of selected application routes without requiring application developers to assemble post-quantum cryptography themselves. Its public integration surface remains small, while its internal contract combines native-runtime trust, cryptographic handshake, session, strict sequence, replay defense, request semantics, protected responses, lifecycle control, and fail-closed behavior.

1. Product Overview

1.1 Definition

ODRE PQC is not a library that merely exposes PQC functions. It is an Application Security Boundary between protected FastAPI routes and a native PQC runtime. Before a protected request reaches the business handler, the product verifies runtime identity, protocol state, session validity, sequence continuity, and the authenticated meaning of the request.

The product covers:

- Installed runtime and artifact identity
- ML-KEM-768 key encapsulation
- ML-DSA-65 digital signatures
- Hybrid handshake and session establishment
- Session expiry and disposal
- Strict sequence and replay blocking
- Method, path, query, header, and body semantic binding
- Protected response metadata and body binding
- Fail-closed termination before handler execution
- Worker, crash, restart, and shutdown lifecycle
- Native-trust differences on Windows and Ubuntu
- Operational diagnosis through [doctor](#), [verify](#), and [status](#)

1.2 Problem Being Solved

The ability to call a PQC algorithm is not the same as the ability to process an application request safely. A deployable product must also answer which runtime was loaded, whether the verified file is the loaded file, how a handshake is bound to a session, whether a replay can execute a handler twice, whether outer HTTP metadata can change business meaning, and whether stale state survives a crash.

ODRE PQC turns these questions into conditions that must be satisfied before a protected handler can run.

1.3 Core Product Proposition

- Integration must remain simple. Application developers do not implement the cryptographic state machine themselves.
- Protected meaning must remain bound end to end. The boundary protects what the application actually processes, not only an encrypted body.
- Unproven safety means no execution. Protected routes do not silently bypass or automatically downgrade to classical-only protection.

1.4 Target Environments

The product targets FastAPI services, protected APIs requiring confidentiality, integrity and replay defense, mixed Windows and Ubuntu fleets, and environments where operators must be able to diagnose runtime state from the command line.

Item	Verified combination
Windows	Windows Server 2022 AMD64 / CPython 3.12.10
Linux	Ubuntu 24.04.4 LTS AMD64 / CPython 3.12.3
Cryptographic runtime	OpenSSL 3.5.8
Key encapsulation	ML-KEM-768
Digital signature	ML-DSA-65
Wire protocol	odre-pqc/3

1.5 Relationship to TLS, VPN, WAF, and PQC Libraries

Component	Primary role	Difference from ODRE PQC
TLS	Channel confidentiality and integrity	ODRE PQC binds application routes, session, sequence, and request meaning.
VPN	Limits network reachability	ODRE PQC validates protected requests even inside an allowed network.
WAF	Filters HTTP patterns and policies	ODRE PQC controls whether a cryptographically valid request may execute a handler.
PQC library	Supplies cryptographic primitives	ODRE PQC composes primitives into a fail-closed application lifecycle.
ODRE PQC	Embedded application boundary	Covers runtime trust through response binding under one execution contract.

These components are complementary layers. ODRE PQC adds application-execution conditions above transport, network, and HTTP filtering.

1.6 Public and Protected Routes

Public routes may remain available according to application policy. Protected routes execute only after runtime, handshake, session, sequence, and request semantics have been verified. If the protection runtime cannot prove a safe state, public health or information routes may remain available while protected routes fail closed. This is Fail-Closed with Public Availability.

2. Minimal Integration Experience

2.1 Installation Flow

```
Install Wheel
→ odre-pqc init
→ odre-pqc doctor
→ odre-pqc verify
→ install(app)
→ Start application
→ odre-pqc status
```

Stage	Purpose
Install	Places the Python package and command-line entry points.
init	Initializes platform runtime and local security state.
doctor	Diagnoses environment, permission, file, and runtime prerequisites.
verify	Runs real PQC self-tests and installation verification.
install(app)	Registers the security boundary in FastAPI.
status	Reads runtime state without mutating it.

2.2 FastAPI Integration Point

```
from fastapi import FastAPI
from odre_pqc import install

app = FastAPI()
install(app)
```

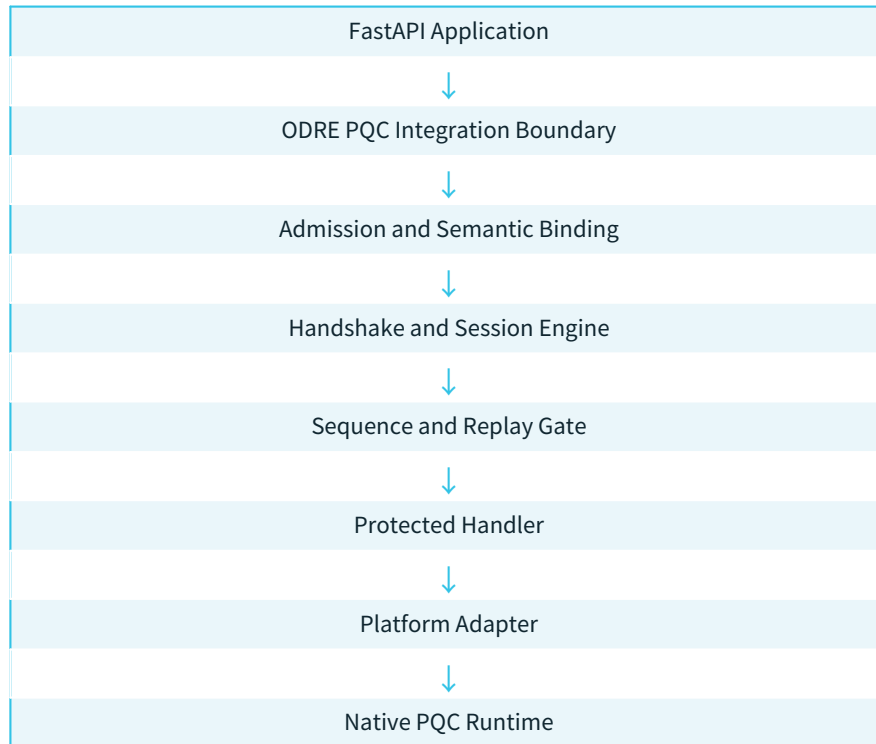
The two-line surface represents separation of responsibility. Application handlers do not manage ML-KEM ciphertext, ML-DSA signatures, replay windows, provider paths, or native handles. The boundary applies the same security order to every protected request.

2.3 Application Responsibilities

The application still selects protected routes, applies user and organization authorization, validates business input, defines data retention, and decides which functions may remain public. ODRE PQC ensures that a protected request has passed the security contract before those policies execute.

3. Product Architecture

3.1 Layer Model



The architecture consists of Application Integration, Admission, Protocol, Semantic Binding, Platform Adapter, and Native Runtime layers. Operating-system differences are absorbed below the common Python security contract.

3.2 Protected Request Pipeline

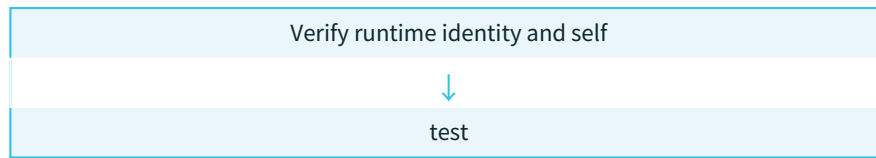
```
Protected request
→ Admission
→ Handshake or valid session
→ Strict next-sequence check
→ Request semantic binding
→ Protected handler exactly once
→ Response binding and protection
```

Order matters. Unsupported streaming, unbounded output contracts, malformed envelopes, and oversized input must be rejected before they can create handler side effects.

3.3 Admission

Admission validates message type, mandatory fields, encoding, aggregate size, duplicate or conflicting fields, supported response forms, and required security metadata. Passing Admission means only that the request can be safely interpreted by the next layer; it does not authenticate the request.

3.4 Handshake and Session



Protocol version, algorithm set, both endpoint materials, transcript, route meaning, and session identifier are bound together. A session is not created if any required relationship cannot be verified.

3.5 Cryptographic Roles

- ML-KEM-768 encapsulates and recovers shared secret material.
- ML-DSA-65 authenticates handshake material and transcript integrity.
- Hybrid Handshake binds the required cryptographic components to the same peer, session, and route semantics.

Successful primitive calls alone do not establish protection. Runtime identity, transcript binding, key confirmation, and session creation must also complete. Failure of a required component does not become a classical-only success.

3.6 Session Lifecycle

Uninitialized → Handshaking → Active → Expired → Closed
↘ validation failure ↗

An active session is not trusted indefinitely. Expiry, runtime-state change, worker-ownership change, sequence violation, or integrity failure closes the protected state.

3.7 Strict Sequence and Replay Defense

Protected requests must carry exactly the next sequence value.

- Duplicate: reject an already processed value
- Rollback: reject a lower value
- Gap: reject a skipped value
- Race: accept only one of concurrent identical values

The accept decision and sequence update are atomic with respect to handler dispatch so the same protected request cannot execute the business handler twice.

3.8 Request Semantic Binding

Encrypting only the body leaves method, path, query, and meaningful headers open to substitution. ODRE PQC binds HTTP method, normalized path, normalized query, security-relevant headers, payload or envelope, session ID, sequence, protocol version, and message type. A mismatch between authenticated meaning and handler-visible meaning prevents execution.

3.9 Response Binding

Status code, content type, protected headers, cookies, response body, and the corresponding request session and sequence are bound into the protected result. A response form that cannot be represented safely is rejected explicitly.

4. Fail-Closed Security Contract

Fail-Closed means that a request whose protection conditions cannot be proved does not enter the protected handler. It does not necessarily mean that every public route or process must stop.

Protected routes are blocked when runtime identity, provider or dependency trust, PQC self-test, handshake transcript, session state, strict sequence, semantic binding, worker ownership, or safe recovery cannot be established. Automatic classical downgrade is not accepted as equivalent protection.

Rejecting a response after a handler has already performed database or external-service side effects is too late. Admission and supported-contract decisions are therefore placed before handler execution whenever possible.

5. Platform Adapter and Native Runtime

5.1 Cross-platform Boundary

Common Python Security Contract

└─ Windows Adapter: DLL, ACL, Junction, Reparse, Loaded Module

└─ Ubuntu Adapter: SO, Permission, Symlink, FD, Inode, Loader

↓

Verified Native Runtime

5.2 Windows Native Trust

Windows trust covers DLL and provider paths, ACL and ownership, junction and reparse bypass, dependency search paths, verified-versus-loaded identity, runtime replacement attempts, and handle/module lifetime during shutdown. A correct pre-load hash is not sufficient if the loader can select a different object or dependency.

5.3 Ubuntu Native Trust

Ubuntu trust covers shared-object and parent permissions, symlink and unsafe-parent attacks, ([st_dev](#), [st_ino](#)) identity, atomic replacement between verification and loading, provider/module/engine/config injection, dependency resolution, and recovery after SIGKILL or restart. Open file descriptors and inode identity bind critical transitions to actual objects instead of mutable path strings.

5.4 Runtime Archive and Manifest

The product verifies platform selection, archive hash, per-file manifest hashes, permissions and parent trust, loaded-object identity, and package/runtime version compatibility. Initialization follows platform identification, manifest selection, staging, object-identity locking, atomic promotion, native load, runtime version check, and real ML-KEM/ML-DSA self-tests. Missing primitives or identity proof cause initialization to fail closed.

6. Operational Diagnosis

- **init**: prepares runtime and local security state while detecting existing, competing, partial, or untrusted initialization.
- **doctor**: diagnoses OS, architecture, Python, runtime identity, permissions, hashes, symbols, providers, dependencies, and state-store access.
- **verify**: checks package/runtime versions, native load, real ML-KEM-768 and ML-DSA-65 self-tests, hybrid handshake, session lifecycle, protected round trip, and absence of automatic downgrade.
- **status**: reports state read-only without changing sessions or leases and without exposing secrets or sensitive paths.

7. Worker and Lifecycle Architecture

Competing workers must not silently share unsupported ownership of runtime or security state. A normal shutdown stops new protected admission, drains bounded in-flight work, disposes sessions and leases, closes native resources, releases locks, and records the stopped state in that order.

Crash, SIGKILL, and power loss cannot rely on cleanup. Restart verifies process ownership, stale locks, sessions, worker leases, status state, and partially initialized runtime before returning to Ready.

```
Starting → Ready → Draining → Stopped
↓ ↓
Blocked Crashed → Recovering → Ready or Blocked
```

8. Trust Boundaries and Attack Surface

Primary trust boundaries exist between the untrusted client and HTTP parser, outer HTTP metadata and authenticated semantics, ODRE PQC and the protected handler, Python and the native loader, path strings and filesystem objects, and worker state and persistent state.

Attack surface	Representative attack	Defense location
Admission	Malformed, oversized, smuggling shape	Admission Gate
Session	Expired or cross-session reuse	Session Engine
Sequence	Duplicate, rollback, gap, race	Atomic Sequence Gate
Request meaning	Method, path, query, header substitution	Semantic Binding
Response meaning	Status, header, cookie substitution	Response Binding
Native files	TOCTOU and atomic replacement	Platform Adapter
Loader	Dependency, provider, config injection	Runtime Trust
Filesystem	Symlink, junction, reparse, unsafe parent	Object Identity
Lifecycle	Stale lease, crash, early shutdown	Lifecycle Controller

ODRE PQC does not replace business authorization, application vulnerability remediation, host hardening, DDoS protection, physical security, data-retention policy, or complete client-device integrity. It secures the boundary by which a protected request enters the application and the runtime trust on which that boundary depends.

9. Deployment Architecture

PQC-capable Client

→ Nginx or Load Balancer

→ FastAPI + ODRE PQC Boundary

→ Protected Handler and Application Data

↳ Platform-specific Native Runtime

Nginx or a load balancer retains its normal routing and transport role. ODRE PQC executes inside the FastAPI application before a protected handler. Multi-instance designs must account for session affinity or shared state, sequence consistency, instance runtime identity, worker leases, rolling restart, retry-induced replay, and separation of public and protected health checks.

Before deployment, operators verify supported OS and architecture, wheel/runtime version and hash, clear [doctor](#) conditions, real-PQC [verify](#) results, public/protected route policy, absence of automatic downgrade, restart and recovery procedure, and secret-free logs and status.

10. Product Identity

Item	Value
Product	ODRE PQC
Version	0.2.9
Official Wheel	odre_pqc-0.2.9-py3-none-any.whl
Wheel SHA-256	A8AFA10EE0AFACEF1C0FAF55FF07B96C722920E5AEE8692F6F026E804F028697
Windows Runtime SHA-256	826303E72A76B10041D26113F465ADD8960825C71F50722D6792174E73F9D12C
Ubuntu Runtime SHA-256	EFAD16698203371D07F25B429F6731B9C0954EEBF5B58A0024789EF0611A78D1
OpenSSL Runtime	3.5.8
PQC	ML-KEM-768 / ML-DSA-65
Protocol	odre-pqc/3
Official OS	Windows Server 2022 AMD64 / Ubuntu 24.04 LTS AMD64

11. Conclusion

ODRE PQC defines the security conditions under which an application handler may execute. Runtime identity, handshake, session, strict sequence, replay defense, request/response binding, and lifecycle are enforced as one contract; a protected request that cannot prove those conditions is stopped before handler execution.

The external workflow remains [init](#), [doctor](#), [verify](#), [install\(app\)](#), and [status](#), while platform-specific native trust and cryptographic protocol state are isolated internally. The product's value is not another algorithm name, but an Application-Embedded Post-Quantum Security Boundary whose allow and block conditions can be explained, operated, and verified.